

Lab 3 — File Inclusion and SQL Injection Security Assessment

Student: Jamiu Abidemi Afolabi

Reg No: C11/26/EHIT/17319

Course: Offensive Security I

Assessment: Lab 3 of 5

Assessment Date: 01//09/2026

Environment: Kali Linux / DVWA / OWASP Mutillidae II

1. Introduction

1.1 Assessment Overview

Briefly describe the purpose of the assessment and the two vulnerability areas tested.

1.2 Scope and Rules of Engagement

State that testing was limited to the authorised local/host-only DVWA and Mutillidae II environments and that only benign test data was used.

1.3 Laboratory Environment

Component	Details
Operating System	Kali Linux
Target 1	DVWA
Target 2	OWASP Mutillidae II
Network	Local/Host-only
Testing Tools	Browser, Burp Suite, terminal and approved tools

1.4 Testing Methodology

Explain briefly that normal behaviour was established first, followed by controlled testing, evidence collection and secure-mode retesting.

2. File Inclusion Assessment

2.1 DVWA File Inclusion

Baseline

Request/URL: http://127.0.0.1:4280/vulnerabilities/fi/?page=file1.php

Controlling Parameter: page

Normal Behaviour: The application loads the selected PHP page through the page parameter.

Evidence: E-02-DVWA-file-inclusion-baseline.

LFI Validation

Benign Marker: local-marker.txt

Marker Content: This is file inclusion marker for lab3's file inclusion engagement

Marker Hash: 6cb828054f5a61f55bb549d01b15c687a984ed24bf7ed9a6c28ef89cdadf4f91

Test Input: /tmp/local-marker.txt

Observed Behaviour: The page parameter was changed from the normal application resource to the benign marker located at **/tmp/local-marker.txt**. The marker content was returned in the DVWA response. PHP warnings also identified **/tmp/local-marker.txt** as the source of the output.

Result: Vulnerable - LFI demonstrated

Evidence: E-03-DVWA-LFI-marker, E-04-Path-Error-Disclosure

RFI Validation

Status: Tested

Marker Resource: remote-marker.txt

Marker Content: This is the test for the RFI vulnerability

Marker SHA-256:

117ba05a69fa42c87a177577f81f21a67d7367f2371f3e4aaf311e24043b219d

Local Marker Service: Python HTTP server running on Kali at **172.18.0.1:8000**

Observed Server-Side Retrieval: The marker server recorded a **GET /remote-marker.txt** request with an **HTTP 200** response after the remote URL was supplied to the DVWA page parameter. This confirms that the application-side process requested the remote resource.

Observed Application Response: The contents of the remote marker were also displayed in the DVWA browser response using the URL:

http://127.0.0.1:4280/vulnerabilities/fi/?page=http://172.18.0.1:8000/remote-marker.txt

Evidence: E-05a-RFI-server-side-retrieval, E-05b-RFI-response-behaviour

2.2 Mutillidae II File Inclusion

Baseline

Request/URL: http://127.0.0.1:8888/index.php?page=arbitrary-file-inclusion.php

Controlling Parameter: page

Normal Resources: arbitrary-file-inclusion.php

Normal Behaviour: The application accepts the **page** parameter and loads the selected file-inclusion functionality normally.

Evidence: E-[XX]

LFI/RFI Validation

Objective: This is to determine whether file inclusion can be controlled through user input.

Benign Marker: local-marker.txt

Observed Behaviour: The page parameter was changed from the normal baseline resource arbitrary-file-inclusion.php to the benign marker at /tmp/local-marker.txt. The web application processed the path supplied and returned the marker content. This indicates that the user-controlled parameter could be used to access a local file outside the intended resource selection.

Result: Vulnerable/LFI demonstrated

Evidence: E-07-Mutillidae-LFI-marker, E-07a-Mutillidae-LFI-marker-copy

RFI Validation

Test Objective: Determine whether file inclusion can be controlled through user input and whether remote resources can be included.

Benign Marker: remote-marker.txt

Marker Content: This is the test for the RFI vulnerability

Marker SHA-256:

117ba05a69fa42c87a177577f81f21a67d7367f2371f3e4aaf311e24043b219d

Test Input: http://172.18.0.1:8000/remote-marker.txt

Observed Behaviour: The page parameter accepted the remote HTTP resource hosted by the Kali marker server. The marker content was displayed within the Mutillidae II response.

The marker server also recorded successful requests from the Mutillidae container, including GET /remote-marker.txt with HTTP status 200. This provides server-side confirmation that Mutillidae retrieved the remote resource.

RFI Status: Tested and demonstrated.

Result: Vulnerable — controlled RFI demonstrated.

Evidence: E-08-Mutillidae-RFI-server-side, E-08b-RFI-response

2.3 File Inclusion Security Comparison

Application	Weak Configuration	Strong Configuration	Result
DVWA	Low - marker was included	Impossible - <code>/tmp/local-marker.txt</code> returned error: file not found	Blocked
Mutillidae II	Security 0 - marker was included	Secured - <code>/tmp/local-marker.txt</code> still successfully retrieved	Not Blocked

Security Control Observed:

The DVWA's impossible security level successfully blocked the supplied arbitrary path in the page parameter from being included. But even with security level 5 on Mutillidae II, the retrieval of the benign marker was still successful during the test. Despite using the highest security level on the web app, the file-include path remained accessible.

3. SQL Injection Assessment

3.1 DVWA SQL Injection

Baseline

Normal Valid Request: id=1

Observed Behaviour: The application processed the supplied user ID and returned the corresponding database record:

- **ID:** 1
- **First name:** admin
- **Surname:** admin

Evidence: E-09-DVWA-SQLi-baseline

SQL Injection Validation

Affected Parameter: id

Test Approach: A harmless boolean-based test was performed using a true condition followed by an equivalent false condition. No data extraction or database enumeration was performed.

True Condition: `1' AND '1'='1`

Observed result: The application returned the same database record observed during the normal valid request:

- **ID:** 1
- **First name:** admin
- **Surname:** admin

False Condition: `1' AND '1'='2`

Observed result: No database record was returned.

Boolean/Behavioural Observation: The assessment indicates that the user-controlled input affected the logic of the database query. The True condition returned the expected database record as the normal input, while the False condition didn't return anything.

Result: Vulnerable - SQL Injection demonstrated

Evidence: E-10-DVWA-SQLi-validation

Authentication/Marker Evidence

Test Type: [Authentication / Marker Record]

Authorised Account/Marker: [Insert]

Observed Result: [Insert]

Evidence: E-[XX]

3.2 Mutillidae II SQL Injection

Baseline

Normal Authentication Request: <http://127.0.0.1:8888/index.php?page=login.php>

Valid Account Test:

Username: admin

Password: admin

Observed Result: Password Incorrect

Invalid Account Test:

Username: test

Password: test

Observed Result: Account does not exist for username: test

Observed Behaviour: Here, we were able to distinguish between an existing username with an incorrect password and a username that doesn't exist. These responses establish the normal authentication behavior before we conduct SQLi on the web application.

Evidence: E-11a-Mutillidae-valid-test, E-11-Mutillidae-invalid-test

SQL Injection Validation

Affected Parameter: username

Test Approach: A harmless boolean-based authentication test was performed against the Mutillidae II login function on the User Data (SQL) page. Testing was limited to demonstrating whether the username input could alter the authentication logic.

Test Input: ' OR '1'='1' #

Observed Behaviour: The application accepted the manipulated username input and authenticated the session as the admin account. The application then displayed the administrator's user profile, confirming that the authentication logic had been bypassed.

Boolean/Behavioural Observation: The successful authentication contrasted with the normal failed-login responses recorded during the baseline tests. This demonstrates that the user-controlled username input was able to change the logic used by the underlying authentication query.

Result: Vulnerable — SQL injection demonstrated with authentication bypass.

3.3 SQL Injection Security Comparison

Application	Weak Configuration	Strong Configuration	Result
DVWA	Low - the input 1' AND '1'='1' returned First name: admin Surname: admin	Impossible - the input 1' AND '1'='1' was blocked and the app returned nothing.	Blocked
Mutillidae II	Security Level 0 - with username= ' OR '1'='1' # and password = ab, it	Secured (Security Level 5) - the same input was rejected at secure level and returned a response saying	Blocked

	logged in as the administrator.	“Dangerous characters detected. We can’t allow these”	
--	---------------------------------	---	--

Security Control Observed:

The DVWA's impossible security level successfully blocked the SQL injection arbitrary input of `1' AND '1'='1` and returned nothing. Also, Mutillidae II's security level 5 also blocked the login details `username= ' OR '1'='1' #` and `password = ab`, and didn't go through with the authentication.

4. Security Analysis

4.1 File Inclusion Analysis

How is server-side file inclusion distinguished from a browser redirect?

Why is a harmless marker sufficient to prove path control?

The harmless marker is sufficient to prove path control because the aim of the lab assessment is to establish whether attacker controlled input can influence the resource selected by the application. Successful retrieval of that known marker through an unintended path demonstrates that the application accepted and processed the supplied resource location without requiring access to sensitive files.

4.2 SQL Injection Analysis

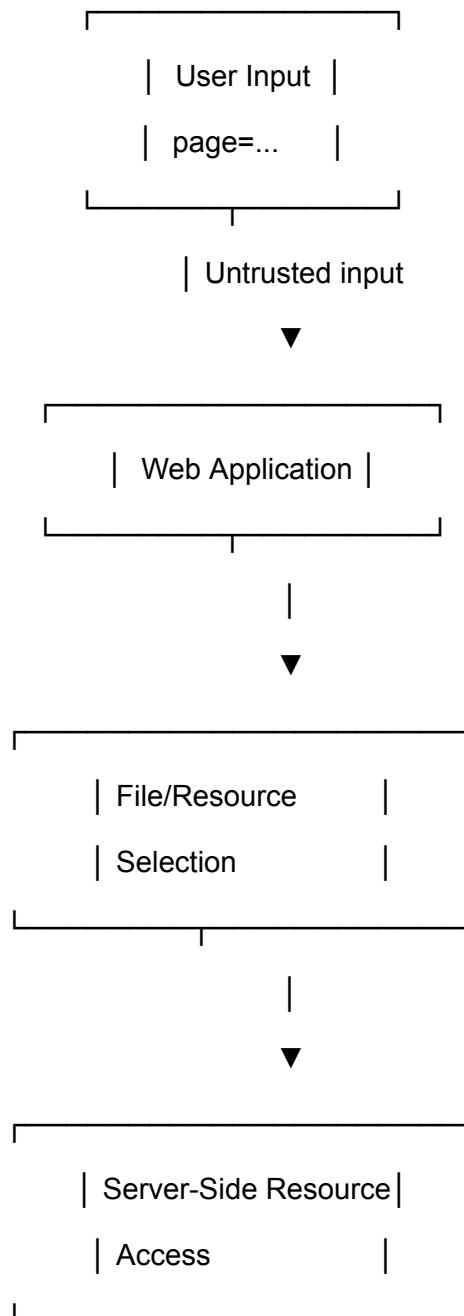
What shows that SQL input changed query logic rather than merely causing an error?

4.3 Trust Boundaries

File Inclusion

The trust boundary occurs where externally supplied **page** or file-selection input is accepted by the application and used to determine which resource is accessed. The security weakness occurs when this untrusted input reaches server-side file access without sufficient restriction.

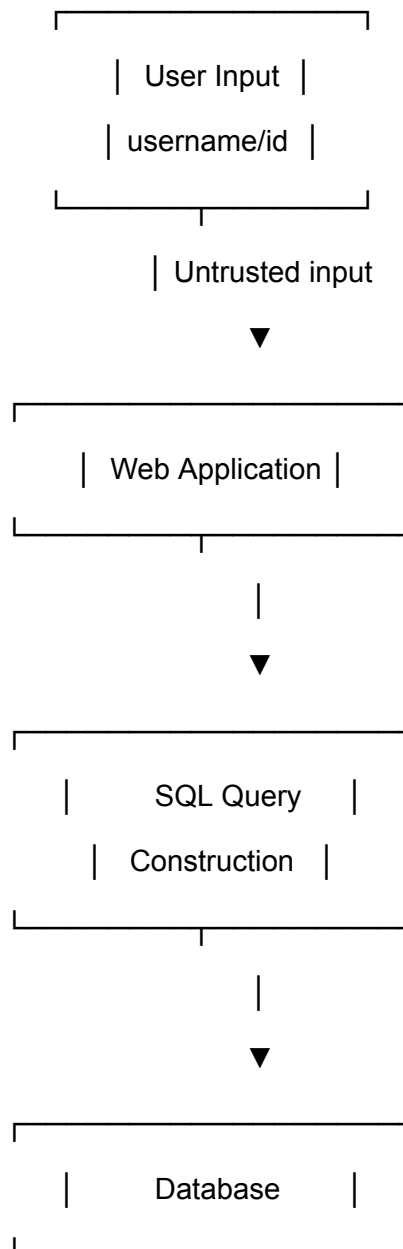
User Input → Web Application → File/Resource Selection → Server-Side File Access



SQL Injection

The trust boundary occurs between untrusted user input and the database query construction/execution layer. The security weakness occurs when user-controlled input is incorporated into SQL logic instead of being treated strictly as data.

User Input → Web Application → SQL Query → Database



5. Remediation

Finding	Recommended Remediation
Unsafe file/resource selection	Use fixed resource mappings or allowlists.
Path traversal/inclusion	Validate and canonicalise paths before access.
Remote file inclusion	Disable unnecessary remote inclusion and restrict outbound access.
SQL Injection	Use prepared statements/parameterised queries.
Database errors	Return generic errors and keep detailed errors in controlled logs.
Excessive database privileges	Apply least-privilege database permissions.

6. Evidence and Cleanup

6.1 Evidence Summary

[Brief summary of the evidence collected.]

See: `evidence-register.md`

6.2 Evidence-Minimisation Statement

In this lab assessment, we only collected the minimum evidence required to demonstrate each finding. There were no database dumps, operating-system secrets, private keys, credentials or unrelated data collected.

6.3 Cleanup and Restoration

- Local marker service stopped: Yes
- Temporary marker files removed: Yes
- DVWA reset: Yes
- Mutillidae II reset: Yes
- Temporary accounts checked: Yes
- Database dump created: **No**
- Final environment state: The final environment state was reset back to its initial state.

7. Conclusion

The Lab 3 assessment was successfully completed and provided practical understanding of how Local File Inclusion (LFI) and Remote File Inclusion (RFI) vulnerabilities work. The assessment also demonstrated how SQL injection can be identified through controlled behavioural testing.

All testing was carried out within the authorised scope of the laboratory environment. Benign markers were used only to validate the identified vulnerabilities, and testing was stopped once sufficient evidence had been obtained. No further data extraction, database dumping or unrelated enumeration was performed.

At the end of the assessment, both web applications and the testing environment were cleaned up. Temporary markers were removed, the local marker service was stopped, and the environment was restored to its intended state.

Appendices

Appendix A — Activity Log

See `activity-log.md`.

Appendix B — Evidence Register

See `evidence-register.md`.